

DST Airlines

Membre de l'équipe

Nicolas CALO
Johan CLOOS
Younes Es-soualhi
Rathana LAT
Youssef ZNATI

Étape 4 - Déploiement

Description

Objectif

- Il faut créer une API pour requêter ces différentes bases de données.
- Faire un conteneur Docker de chaque composant du projet (BDD, API) et faire un docker-compose fonctionnel.

Modules / Masterclass / Templates

- FastAPI / Flask / Dash / Streamlit
- Docker

Conditions de validation du projet

- API FastAPI
- Conteneur, Fichier yaml du docker-compose

Module FastAPI / Dash

L'interface utilisateur, créé sous Dash, permet de choisir les critères avec des listes déroulantes en filtrant :

- Continent
- Sous continent
- Pays
- Région
- Aéroport de départ
- Compagnie aérienne
- Flight_id (composé de la date de départ, du numéro de vol, du code IATA de départ et d'arrivé)

Continent:
Europe

Subcontinent:
Western Europe

Country:
France

Location:
Ile-de-France

Departure airport:
ORY - Aéroport de Paris-Orly
CDG - Paris Charles de Gaulle Airport

20251121+AF+0648ORYRUN

Demande de prédiction

Errors Callbacks | v3.2.0 Dash update available - v3.3.0 | Server >>

L'interface utilisateur inclut également un module FastAPI utilisant la méthode de requête POST pour envoyer une demande de prédiction sur le retard du vol sélectionné.

La connexion à la base de données Postgresql depuis Dash se fait avec la fonction `create_engine` de la librairie SQLAlchemy.

```
def get_data():
    query = "select distinct(flight.flight_id), \
        Continent_name, \
        Subcontinent_name, \
        Country_name, \
        Location_name, \
        Airport_name, \
        flightLegs_depInfo_airport_code, \
        flightLegs_depInfo_airport_places_depPosTerm_boardingTerminal, \
        flightLegs_depInfo_airport_places_depPosTerm_gateNumber, \
        flightLegs_depInfo_times_scheduled_date, \
        flightLegs_depInfo_times_scheduled_time, \
        flightLegs_depInfo_times_scheduled_year, \
        flightLegs_depInfo_times_scheduled_month, \
        flightLegs_depInfo_times_scheduled_day, \
        flightLegs_depInfo_times_scheduled_hour, \
        flightLegs_depInfo_times_scheduled_minute, \
        flightLegs_depInfo_times_number_week, \
        flightNumber, \
        airline_code, \
        airline_name, \
        flightStatusPublic, \
        flightLegs_aircraft_typeCode, \
        flightLegs_scheduledFlightDuration, \
        flightLegs_serviceType, \
        flightLegs_aircraft_ownerAirlineCode, \
        flightLegs_status, \
        delay_status, \
        flightLegs_irregularity_delayDuration, \
        flightLegs_irregularity_delayInfo_delayReasonPublicLong, \
        flightLegs_irregularity_delayInfo_delayReasonPublicShort \
    from Continent Continent \
    INNER JOIN Subcontinent Subcontinent ON Continent.Continent_ID = Subcontinent.Continent_ID \
    INNER JOIN Country Country ON Country.Subcontinent_ID = Subcontinent.Subcontinent_ID \
    INNER JOIN Location Location ON Location.Country_ID = Country.Country_ID \
    INNER JOIN Airport Airport ON Airport.Location_ID = Location.Location_ID \
    INNER JOIN departure_airport departure_airport ON departure_airport.flightLegs_depInfo_airport_code = airport.Iata_code \
    INNER JOIN flight flight ON flight.flight_id = departure_airport.flight_id \
    INNER JOIN delay delay ON delay.flight_id = flight.flight_id \
    where flightLegs_depInfo_times_scheduled_date > current_date;"

    df = pd.read_sql(query, engine)
    return df
```

Exemple d'un vol sélectionné pour un départ de l'aéroport d'Orly et de Charles de Gaulles.

Départ d'Orly :

id

Dash

×

+

—

□

×

←

↺

🏠

🕒 127.0.0.1:8050

☆

⌵

InPrivate (2)

⋮

Continent:

Europe

×

▼

Subcontinent:

Western Europe

×

▼

Country:

France

×

▼

Location:

Ile-de-France

×

▼

Departure airport:

ORY - Aéroport de Paris-Orly

×

▼

Departure airline:

AF - AIR FRANCE

×

▼

Departure flight_id:

20251121+AF+0648ORYRUN

×

▼

Demande de prédiction

🚩 Errors

⌘ Callbacks

v3.2.0

Dash update available - v3.3.0

Server 🟢

⏏

Départ de Charles de Gaulles :

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8050'. The page title is 'Dash'. The interface contains several dropdown menus for selecting flight parameters:

- Continent:** Europe
- Subcontinent:** Western Europe
- Country:** France
- Location:** Ile-de-France
- Departure airport:** CDG - Paris Charles de Gaulle Airport
- Departure airline:** AF - AIR FRANCE
- Departure flight_id:** 20251121+AF+0002CDGJFK

Below these dropdowns is a button labeled 'Demande de prédiction'. At the bottom of the interface, there is a status bar with the following elements:

- Errors** (with a warning icon)
- Callbacks** (with a gear icon)
- Version: **v3.2.0**
- Update button: **Dash update available - v3.3.0**
- Server status: **Server** (with a green checkmark icon)
- Refresh button (with a circular arrow icon)

Code d'état des deux requêtes depuis l'interface utilisateur :

```
ubuntu@ip-172-31-36-159:~/dash$ python3 scriptDash9.py
Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'scriptDash9'
* Debug mode: on
Europe, Western Europe, France, Ile-de-France, ORY - Aéroport de Paris-Orly, AF - AIR FRANCE, 20251121+AF+06480RYRUN
<Response [200]>
Europe, Western Europe, France, Ile-de-France, CDG - Paris Charles de Gaulle Airport, AF - AIR FRANCE, 20251121+AF+0002CDGJFK
<Response [200]>
```

Code d'état des deux requêtes (actuellement en local) vers le serveur distant :

```
ubuntu@ip-172-31-36-159:~/dash$ uvicorn main_api:api --reload
INFO: Will watch for changes in these directories: ['/home/ubuntu/dash']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [6545] using StatReload
INFO: Started server process [6547]
INFO: Waiting for application startup.
INFO: Application startup complete.
Europe, Western Europe, France, Ile-de-France, ORY - Aéroport de Paris-Orly, AF - AIR FRANCE, 20251121+AF+06480RYRUN
INFO: 127.0.0.1:35464 - "POST /flight HTTP/1.1" 200 OK
Europe, Western Europe, France, Ile-de-France, CDG - Paris Charles de Gaulle Airport, AF - AIR FRANCE, 20251121+AF+0002CDGJFK
INFO: 127.0.0.1:44190 - "POST /flight HTTP/1.1" 200 OK
```

Le serveur distant est celui qui contient les modèles de machine learning pour renvoyer vers l'interface utilisateur la prédiction du retard avec la méthode de requête POST.

Conclusion Dash / FastAPI

L'interface utilisateur développée avec Dash permet de sélectionner les vols avec des listes déroulantes en cascade pour demander la prédiction du retard vers le serveur du machine learning avec la méthode de requête POST développée avec FastAPI .

Il reste à développer sur l'interface utilisateur, la réception du POST venant du machine learning avec la prédiction du retard pour l'affichage et une image docker pour Dash.

API Prédiction Machine Learning

GET

/model_parameters_and_metrics

Retrieve The Prediction Model Training Parameters And Validation Metrics

⌵

Retrieve the prediction model training parameters and test metrics of the current best models for status and delay

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \nhttp://127.0.0.1:8001/model_parameters_and_metrics' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8001/model_parameters_and_metrics

Server response

Code

Details

200

Response body

{\n \"pipeline\": \"Logistic OVR\",\n \"timestamp\": \"2025-11-20T16:01:13\",\n \"best_pipeline\": true,\n \"mode\": \"simple\",\n \"problem_type\": \"classification_status\",\n \"dataset_size_training\": 463110,\n \"dataset_size_testing\": 115780,\n \"accuracy\": 0.8507341509759889,\n \"macro_avg_precision\": 0.5548654679671609,\n \"macro_avg_recall\": 0.569623832151331,\n \"macro_avg_f1\": 0.5608843979802117,\n \"processing_time\": 9,\n \"numeric_features\": [],\n \"categorical_features\": [\"flightlegs.aircraft.ownerairlinecode\", \"flightlegs.aircraft.typecode\", \"flightlegs.arrinfo.airport.city.country.areacode\", \"flightlegs.arrinfo.airport_code\", \"flightlegs.depinfo.airport.city.country.areacode\", \"flightlegs.depinfo.airport_code\", \"flightlegs.scheduledflightduration\", \"flightlegs.season\", \"flightlegs.arrinfo.times.scheduled.dayPeriod\", \"flightlegs.depinfo.times.scheduled.dayPeriod\"],\n \"target_variable\": \"flightlegs.publishedstatus\",\n \"estimator_c\": 1,\n \"estimator_dual\": false,\n \"estimator_fit_intercept\": true,\n \"estimator_intercept_scaling\": 1,\n \"estimator_max_iter\": 1000,\n \"estimator_multi_class\": \"deprecated\",\n \"estimator_penalty\": \"l2\",\n \"estimator_solver\": \"lbfgs\",

Download

Response headers

content-length: 3905\ncontent-type: application/json\ndate: Thu, 20 Nov 2025 19:30:05 GMT\nserver: uvicorn

POST

/get_delay_predictions

Get Flight Delay Predictions

"Interrogates the best status (ONTIME, LATE, CANCELLED) and delay models (delay duration in minutes) based on the flight parameters parameters. Will only perform delay duration prediction if the flight is predicted to be late, and will otherwise output NA

Parameters

No parameters

Cancel

Request body

required

application/json

Edit Value

Schema

```
{
  "flightlegs_depinfo_times_scheduled": "2025-12-20T11:40:00.000+00:00",
  "flightlegs_depinfo_airport_city_country_area": "N",
  "flightlegs_depinfo_airport_city_country_code": "NL",
  "flightlegs_depinfo_airport_city_country_name": "NETHERLANDS",
  "flightlegs_depinfo_airport_code": "AMS",
  "flightlegs_depinfo_airport_location_latitude": 52.3083,
  "flightlegs_depinfo_airport_location_longitude": 4.7681,
  "flightlegs_depinfo_times_latestpublished": "2025-12-20T09:35:00.000+01:00",
  "flightlegs_depinfo_times_scheduled": "2025-12-20T09:35:00.000+01:00",
  "flightlegs_legstatuspublic": "SCHEDULED",
  "flightlegs_publishedstatus": "ONTIME",
  "flightlegs_scheduledflightduration": "PT11H10M",
  "flightlegs_servicetype": "J",
  "flightlegs_servicetypename": "Normal Service",
  "flightlegs_status": "S",
  "flightlegs_statusname": "Scheduled",
  "flightnumber": 601,
  "flightstatuspublic": "SCHEDULED",
  "id": "20251220-KL-0601"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/get_delay_predictions' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": "6915fe0fd9897ad5f1e39ede",
    "airline_code": "KL",
    "airline_name": "KLM ROYAL DUTCH AIRLINES",
    "depinfo_airport_id": 1105,
    "depinfo_airport_name": "Amsterdam Airport Schiphol",
    "depinfo_continent_id": 4,
    "depinfo_continent_id-2": 4,
    "depinfo_continent_name": "Europe",
    "depinfo_country_code": "NL",
    "depinfo_country_id": 168,
    "depinfo_country_id-2": 168,
    "depinfo_country_name": "Netherlands",
    "depinfo_icao_code": "AMS",
    "depinfo_icao_code": "EHAM",
    "depinfo_latitude": 52.3086,
    "depinfo_location_id": 759,
    "depinfo_location_id-2": 759,
    "depinfo_location_name": "Noord-Holland",
    "depinfo_longitude": 4.76389,
    "depinfo_subcontinent_id": 16,
    "depinfo_subcontinent_id-2": 16,
    "depinfo_subcontinent_name": "Western Europe",
  }'
```

Request URL

http://127.0.0.1:8001/get_delay_predictions

Server response

Code

Details

200

Response body

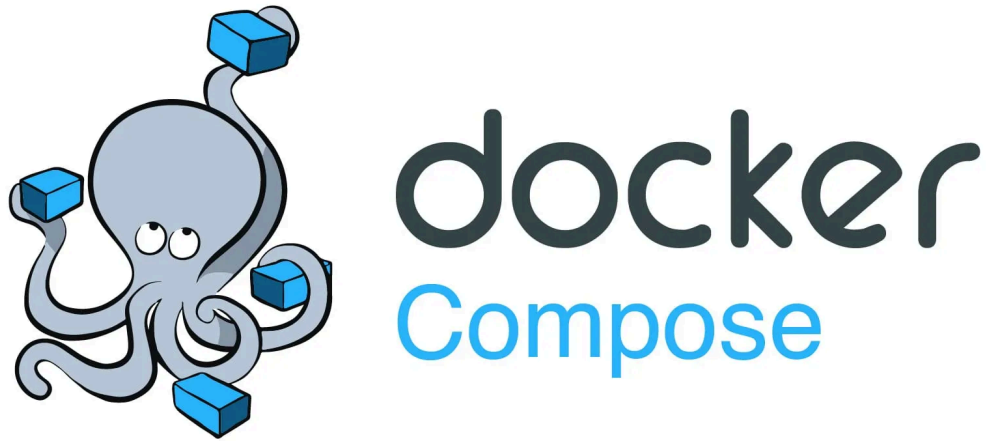
```
{
  "predicted_flightleg_status": "ONTIME",
  "predicted_delay_min_classification": "NA",
  "predicted_delay_min_regression": "NA"
}
```

Response headers

```
content-length: 119
content-type: application/json
date: Thu, 20 Nov 2025 19:30:33 GMT
server: uvicorn
```

Download

Docker COMPOSE:



Docker nous permet le déploiement de nos bdd, api, dashboards... Aisément, et nous évitant toute problématique de compatibilité du passage d'une machine de dev à une machine de déploiement (serveurs).

Nous proposons un docker compose à la racine du projet donnant le choix à un utilisateur de lancer tout le projet en local, ou de le déployer facilement sur un serveur.

Nous publions de même des images de nos api, dashboards etc... Sur github registry, ou bien docker hub.

version: '3.8'

services:

ml_prediction_api:

build:

dockerfile: ./3_ML/afklm_ml_prediction_api_Dockerfile

container_name: dst_de_airlines_ml_prediction_api

ports:

- "8001:8001"

restart: unless-stopped

healthcheck:

test: ["CMD", "curl", "--fail", "http://127.0.0.1:8001/health"]

interval: 10s

timeout: 5s

retries: 5

start_period: 15s

mongodb:

image: mongo:7.0

container_name: dst_de_airlines_mongodb

environment:

MONGO_INITDB_ROOT_USERNAME: airlines

MONGO_INITDB_ROOT_PASSWORD: airlines

volumes:

- ./bdd_afklm_volumes:/data/db

ports:

- "27017:27017"

healthcheck:

test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test --quiet | grep 1

interval: 10s

timeout: 10s

retries: 5

start_period: 10s

restart: unless-stopped

mongodb_api:

build: ./bdd/MongoDb/dst_de_airlines_api/

container_name: dst_de_airlines_mongodb_api

ports:

- "8000:8000"

environment:

-

MONGODB_URI=mongodb://airlines:airlines@host.docker.internal:27017/airlines?authSource=admin

- DATABASE_NAME=airlines

- FOLDER_PATH=/app/data

extra_hosts:

- "host.docker.internal:host-gateway"

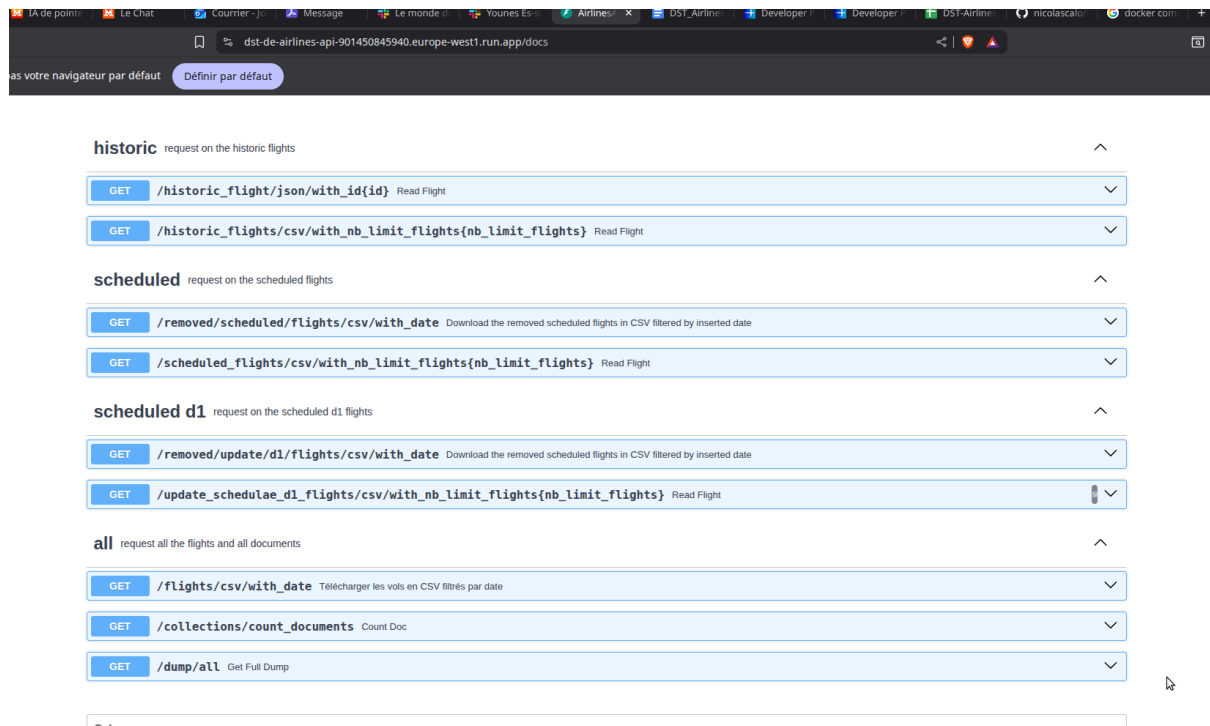
depends_on:

mongodb:

condition: service_healthy

restart: unless-stopped

API MONGO DB:



ROUTE API

Pour rappel: Nous avons divisés les vols en collection:

historic flights est la collection des vols historiques. Qu'est ce qu'un vol historique?

Un vol qui est passé.

Scheduled flights est la collection des vols qui sont programmés et qui n'ont pas encore eu lieu.

update scheduled d1 flights est la collection des vols programmés un jour avant le départ.

Concrètement: Lorsqu'un vol est programmé, et qu'il est à moins de 24 h d'avoir lieu, dans ce cas, ce vol passe dans la collection d1 et disparaît de la collection scheduled. Et quand il a eu lieu, alors il passe dans la collection historique.

Tout cela est maintenu à jour par un script, qui récupère les données extraites de l'api dst airlines.

La bonne pratique aurait voulu que le script requête l'api airlines de airfrance pour insérer directement les données dans notre base de données mongo db, mais faute de moyen, et par l'utilisation de service gratuit, nous stockons les données sur Github (bien que non reconnu comme bonne pratique). Nous le faisons par sécurité afin d'avoir la certitude de ne pas perdre nos données car par l'utilisation gratuite de certains service, nous n'avons pas la garantie du maintien du déploiement de nos bdd ainsi que de la données qui peu se trouver à l'intérieur.

A ce point, les routes restent à **améliorer**. On est bien dans un cas, où le manque de rigueur sur les routes devrait entraîner une recette refusée, et faire suite à une hotfix si on avait travaillé avec git flow, pour réparer les erreurs sur les noms de routes et de plus, uniformiser les noms de routes, faire les retours de recettes...

La définition des noms de routes pourrait se faire avec le client, ou du moins un Product Owner (Po) qui fait l'intermédiaire entre le client et le développement.

De plus, il est très important que lorsque les routes sortent, qu'elles soient conformes, car dans un projet, plusieurs clients peuvent utiliser les routes, et si elles changent, cela coûte de faire une campagne pour modifier les routes ce qui peut être coûteux et lourd. Les types d'erreurs qu'on a ici ne devraient surtout pas passer en recette.

Normalement, les routes devraient être assez explicites pour que le client comprenne ce qu'elles font en les lisant.

Il existe aussi les routes d'API Rest et moins connus et utilisés CQRS.

Cependant, les routes ayant encore besoin d'être travaillées, je vous propose quelques explications et quelques exemples de routes API expliqués.

Information générale sur les routes:

Concernant les routes avec **{nb_limit_flights}**, à l'heure présente, elles semblent ne plus être utiles au client. Cependant, il faudrait déterminer si lorsqu'on entre un nombre de flights, est-ce que ça nous retourne des vols triés sur l'id, sur un autre champ? Sur quel champ il serait sensé de faire un order by? Et surtout comment indiquer clairement dans la doc API comment les vols sont triés.

Les routes avec **with_date**: A la demande du client, une date d'insertion dans la BDD a été ajoutée aux données.

Le client ayant pour projet de ne pas récupérer toutes les données à chaque fois pour synchroniser sa base, il demande de pouvoir rentrer une date pour récupérer que les données ayant été ajoutées après la date qu'il souhaite.

/historic_flight/json/with_id{id}: permet au client de récupérer un flight par son id en json.

/historic_flights/csv/with_nb_limit_flights{nb_limit_flights}: Permet au client d'obtenir le nombre qu'il souhaite de flights de la collection **historic_flights**.

/removed/scheduled/flights/csv/with_date: Retourne les flights qui ont été supprimés de la collection **scheduled**. Route qui avait été demandée par le client, car il a pour projet de synchroniser plus aisément sa bdd avec la bdd Mongo DB dont cette API permet l'interaction.

Ainsi, l'API lui renvoie les vols qui ont été supprimés pour pouvoir les supprimés aussi chez lui.

/scheduled_flights/csv/with_nb_limit_flights{nb_limit_flights}: Permet au client d'obtenir le nombre qu'il souhaite de flights de la collection scheduled_flights.

/flights/csv/with_date: Reccupère tous les flights à partir d'une date données.

/collections/count_documents: Compte le nombre de documents par collections.

Concernant les erreurs, si la connection avec la BDD ne se fait pas, une erreur 503 est envoyé.

Lien de l'api déployé:

https://dst-de-airlines-api-901450845940.europe-west1.run.app/docs#/all/count_doc_collections_count_documents_get

Environnement GCP

Notre architecture s'appuie sur Google Cloud Platform (GCP) pour assurer l'extraction, le stockage et le traitement automatisé des données provenant de l'API Air France. Elle combine des services serverless, du stockage objet et des conteneurs afin de garantir une solution flexible, scalable et maintenable dans un contexte de production.

1. Stockage des données – Google Cloud Storage

Google Cloud Storage (GCS) est utilisé pour centraliser l'ensemble des données brutes collectées depuis l'API Air France. Les fichiers sont stockés dans des buckets GCS et deviennent la source de référence pour tous les traitements ultérieurs.

Rôle dans l'architecture :

- Point d'entrée du stockage des données
- Base de travail pour les étapes de transformation

2. Traitements automatisés – Cloud Run

Le cœur du traitement repose sur Cloud Run, qui exécute des services conteneurisés de façon entièrement serverless.

a. Fonctions Cloud Run

Ces fonctions orchestrent différentes étapes du pipeline :

- extraction des données depuis l'API Air France et dépôt dans Google Cloud Storage
- transformation des données et alimentation des bases de données

b. Cloud Run

2 API FastAPI déployées sur Cloud Run qui servent à:

- Constituer une interface entre les traitements et les données hébergées dans la base NoSQL.
- requêter le modèle machine learning.

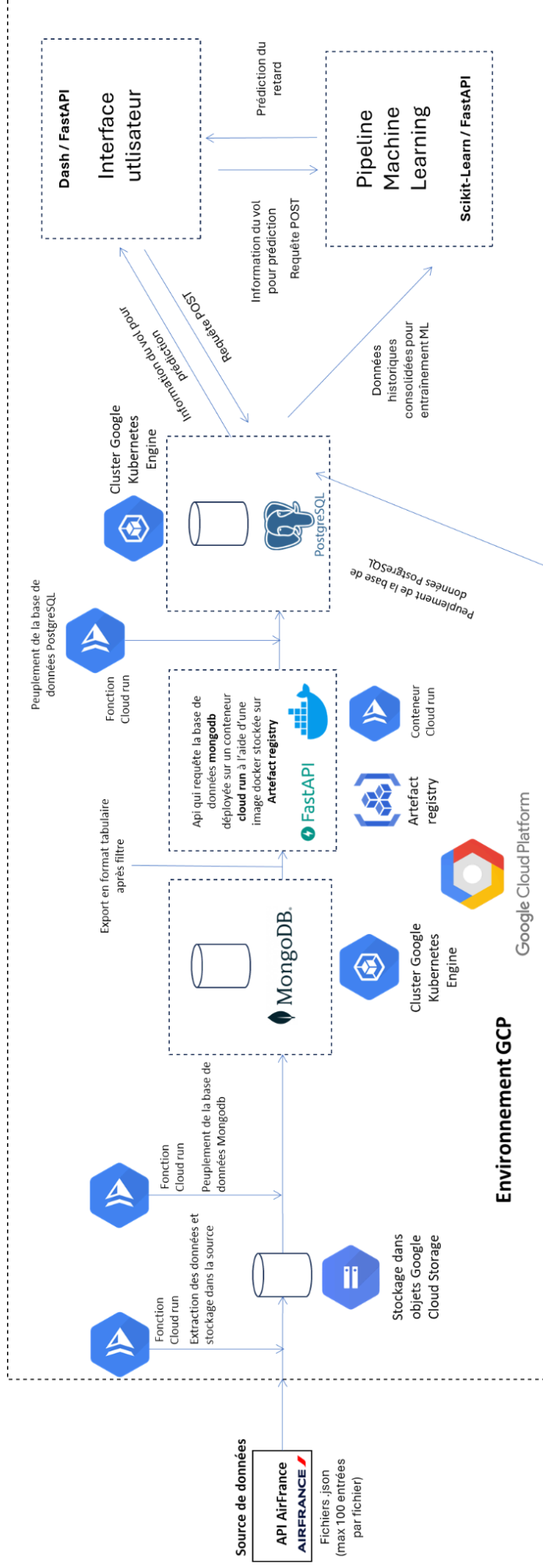
une application Dash.

3. Gestion des images – Artifact Registry

Artifact Registry stocke les images Docker nécessaires aux différents composants Cloud Run. Il assure un stockage sécurisé, une gestion des versions et une intégration directe avec les services de déploiement GCP.

4. Orchestration avancée – Google Kubernetes Engine (GKE)

L'architecture comprend deux clusters Google Kubernetes Engine, utilisés pour héberger les bases de données MongoDB et PostgreSQL.



Fichier .csv issu du Web
scrapping de Wikipedia